

ExaGEMM: Exploration Framework for CPU-Driven ML Inference via Associative In-Register Computing for Low-Bit GEMM

Abstract

Low-bit GEMM is increasingly central to efficient ML inference, yet very-low-bit execution remains a poor fit for conventional CPUs. Practical deployment spans fragmented regimes—from 1/2/4-bit weights to varying activation precision—whose feasibility, reuse opportunity, and support cost differ under fixed SIMD and register-file budgets, making lightweight CPU support selection a first-class design problem. We present **ExaGEMM**, a workload-aware co-design and exploration framework for CPU-native low-bit GEMM via register-resident LUT execution. The key insight is that existing SIMD datapaths already cover table generation and accumulation; the only new hardware is an in-register select/feed mechanism with explicitly modeled cost. ExaGEMM co-explores parameterized kernels and lightweight SIMD ISA support using analytical models of register feasibility, compute cost, memory traffic, and hardware overhead, pruning the candidate space by **99.2%** before simulation. It then identifies non-dominated support points and generates ISA specs, gem5 patches, and GEMM kernels for validation. Across representative ML models and CPU targets, ExaGEMM improves latency by **13.29×** over software-only baselines, while showing that workload-aware frontier selection is especially important for mixed-precision LLM workloads.

1 Introduction

Machine learning (ML) inference is increasingly shaped by low-precision matrix multiplication. Across language, vision, and multimodal workloads, quantization reduces model size [6, 9], memory traffic [15, 25], and arithmetic cost, making low-bit GEMM a central deployment kernel. Once weights are pushed to 4-, 2-, or 1-bit precision, efficient execution depends on how low-bit symbols are packed, indexed, accumulated, and mapped onto the target processor—a problem compounded by growing demand for private, offline, and cost-sensitive on-device inference [21, 26]. For many such systems the target is a CPU [16, 24]: separate GPUs, NPU, or FPGAs introduce area, power, and integration costs that CPU-first or tightly integrated deployments cannot absorb [17].

Very-low-bit GEMM is, however, a poor fit for conventional CPU execution. Table-driven SIMD methods reduce arithmetic cost by replacing explicit low-bit MACs with a compact activation-derived lookup [22], but shift the bottleneck to table construction and repeated movement through the memory hierarchy [14]. A deeper difficulty is that low-bit deployment is no longer concentrated around a single operating point [14, 18]: practical inference spans W1A8 [19], W2A8 [10, 20], W4A8 [8], W4A16 [7], and mixed layer-wise configurations [3, 4, 12, 23], each imposing different pressures on table size, accumulation structure, register usage, and implementation cost. Under a fixed SIMD and register-file budget, support that is feasible and effective for one regime can become wasteful, over-specialized, or infeasible in another. A single fixed kernel or narrowly tuned ISA extension struggles to cover this fragmented space, and selecting *which* CPU support points to design and implement has not been treated as a first-class design problem.

Existing approaches leave this support-selection problem undressed in complementary ways. Software SIMD kernel methods such as T-MAC [22] and Vec-LUT [14] execute on stock CPUs without ISA changes and target fixed precisions, so the hardware support itself is not a design variable. SIMD ISA-extension methods such as T-SAR [16] do propose microarchitectural support, but commit to a single regime—ternary LLM—leaving broader 1/2/4-bit and mixed-precision ML workloads unsupported. Neither category answers the question a hardware designer faces before tapeout: *across a fragmented workload spanning multiple quantization regimes and implementation-cost constraints, **which support points are worth implementing in vector CPU designs, and when is one aggressive fixed design no longer enough?***

We present **ExaGEMM**, a workload-/budget-aware RTL/ISA/kernel co-design and exploration framework for register-resident LUT-based low-bit GEMM on CPUs. The key insight is that table-driven GEMM can largely reuse existing SIMD addition, dot-product, and accumulation datapaths [1, 13]; the main new hardware is an in-register select mechanism that applies lookup tables to staged weight blocks. Because its cost varies with precision, packing granularity, fan-in, tile shape, and mapping style, support cost becomes an explicit and rankable co-design objective.

ExaGEMM analytically co-explores parameterized GEMM kernels and lightweight ISA support using models of register feasibility, instruction count, memory traffic, compute cost, and hardware overhead. This exploration prunes the candidate support space by **99.2%** prior to gem5-based evaluation, since detailed cache- and system-level effects are too costly to resolve across the full search space. ExaGEMM then identifies feasible non-dominated support points and materializes them as RTL datapath designs, ISA specifications, gem5 ISA-model patches, and GEMM kernels for system-level validation. In evaluation across representative vision transformer (ViT) and LLM models and vector CPU targets, ExaGEMM improves latency by **1.92–13.29×** over software-only SIMD baselines, and shows that workload-aware frontier selection is especially important for mixed-precision LLM workloads.

The contributions of this paper are as follows:

- We formulate low-bit CPU GEMM acceleration as a *support-selection* problem and propose a parameterized execution space that unifies mapping choices across multiple weight and activation precisions, covering both uniform quantization and inter-layer mixed-precision models.
- We define a lightweight SIMD-slice support abstraction in which the key new hardware is an in-register select mechanism whose cost scales with precision, packing, fan-in, and tile shape, making hardware overhead an explicit and rankable co-design objective alongside compute cost and memory traffic.
- We present an end-to-end analytical exploration flow that prunes the candidate support space, identifies the non-dominated support frontier, and generates ISA specifications, gem5 ISA-model patches, and GEMM kernels, validated through microarchitectural simulation across CPU targets based on x86 and ARM ISAs.

2 Background and Motivation

Consider a dot product between low-bit weights and activations. In a table-driven block of size e , the e weights are encoded as a symbol $\gamma \in [0, 2^{qe})$, while the current activation group generates a LUT

$$T_a[\gamma] = \sum_{j=0}^{e-1} \text{dec}_q(\gamma, j) a_j, \quad (1)$$

where $\text{dec}_q(\gamma, j)$ is the j -th q -bit weight encoded in γ . Instead of issuing e explicit low-bit MACs, the block contribution is produced by lookup and accumulation. LUT-based low-bit execution trades repeated low-bit arithmetic for a compact activation-derived table.

As sketched by the inset of Fig. 1, this execution style can remain CPU-native. Table generation and application reuse the existing SIMD datapath, while lightweight additional support keeps LUT state in-register and applies it to staged weight blocks. This makes table-driven execution attractive for CPUs, but also introduces tradeoffs absent in conventional dense GEMM.

The first tradeoff is table size. Increasing e improves arithmetic compression, but also grows the table as 2^{qe} . The second is reuse: reusing a LUT across more rows, columns, or inner products amortizes table-generation cost, but increases register-resident state, aggregation overhead, and pressure on intermediate-result movement. The third is implementation cost: more aggressive support can improve compute efficiency, but requires more hardware state and more specialized logic. Low-bit execution is not merely a lower-precision version of standard SIMD MAC; it is a mapping problem that trades off compression, reuse, state, and hardware cost.

Figure 1 shows why these tradeoffs become a support-selection problem. Practical deployment spans multiple quantization regimes and model mixes, imposing different pressures on reuse, state, and implementation cost. Consequently, no single support point is uniformly best across changing bitwidths, activation precision, and target resource budgets. Figure 1(b) summarizes this through compact, balanced, and aggressive support classes, while Figure 1(c) emphasizes the design goal of selecting an implementation-worthy subset from a larger constrained design space.

Prior CPU-oriented methods demonstrate useful operating points [5, 16, 22, 24], but mostly as regime-specific solutions. They show that table-driven or register-resident low-bit execution can be effective on CPUs, yet they do not answer the broader design question raised by the figure: when the target workload spans multiple low-bit GEMM regimes under explicit implementation-cost constraints, which lightweight CPU support points should actually be implemented? As highlighted in Fig. 1, the candidate support space is too large for exhaustive system-level evaluation, making support selection itself a first-class design problem.

This observation motivates the formulation used in ExaGEMM. Let Ω denote the candidate space of lightweight CPU support points together with their associated kernel mappings. Rather than searching for one best low-bit kernel, ExaGEMM evaluates feasible points $x \in \Omega$ under workload and hardware constraints, prunes the space analytically, and identifies an implementation-worthy non-dominated subset that balances compute efficiency against hardware overhead, as illustrated by the rightmost panel of Fig. 1. The next section makes this concrete by defining the parameterized execution space and analytical models for workload-aware support exploration and selection.

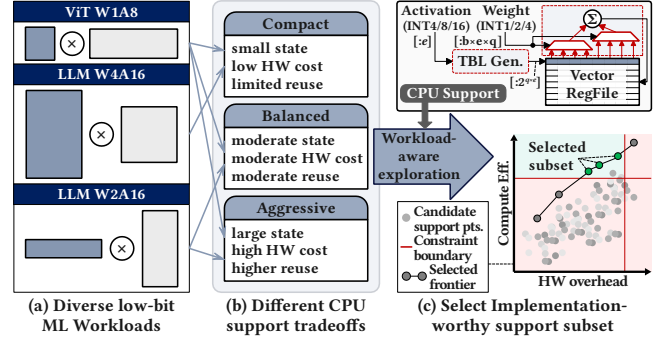


Figure 1: Background and motivation for ExaGEMM. Top right block: lightweight CPU support for table-driven low-bit GEMM, where current activation groups generate in-register LUT state that is applied to staged low-bit weight blocks while reusing the SIMD datapath. (a) Diverse low-bit workloads and quantization regimes impose different pressures on table size, reuse, and register-resident state. (b) These pressures lead to different CPU-support tradeoffs, ranging from compact to more aggressive support with higher reuse and higher hardware cost. (c) ExaGEMM therefore performs workload-aware support exploration and selects an implementation-worthy subset of lightweight CPU support points rather than relying on one fixed design point.

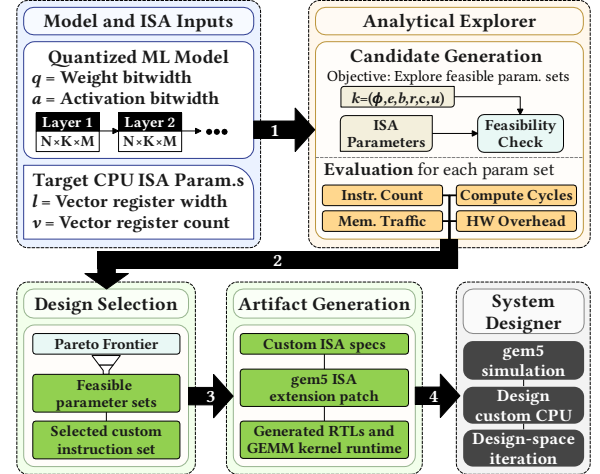


Figure 2: Framework overview of ExaGEMM. Given a quantized ML model and target CPU parameters, ExaGEMM explores feasible parameterized kernels $k = (e, b, r, c, u)$ and evaluates them analytically in terms of instruction count, compute cycles, memory traffic, and hardware overhead. The framework then identifies feasible custom instruction support points, reports the non-dominated support frontier, generates implementation artifacts including ISA specifications, gem5 ISA model patches, and GEMM kernels, and enables designer-guided system-level design-space iteration.

3 Lightweight ISA Support and Analytical Co-Exploration

Figure 2 summarizes the overall ExaGEMM flow. Given the lightweight SIMD support point and parameterized execution space, ExaGEMM uses an analytical surrogate to estimate register feasibility, instruction count, memory traffic, compute cost, and hardware overhead for each candidate kernel. These estimates are used to prune infeasible candidates, select layer-wise kernels, and report a non-dominated support frontier before system-level validation.

3.1 ISA Support and Execution Space

ExaGEMM is built on two ideas: lightweight ISA support and parameterized execution. Low-bit table-driven GEMM can be supported

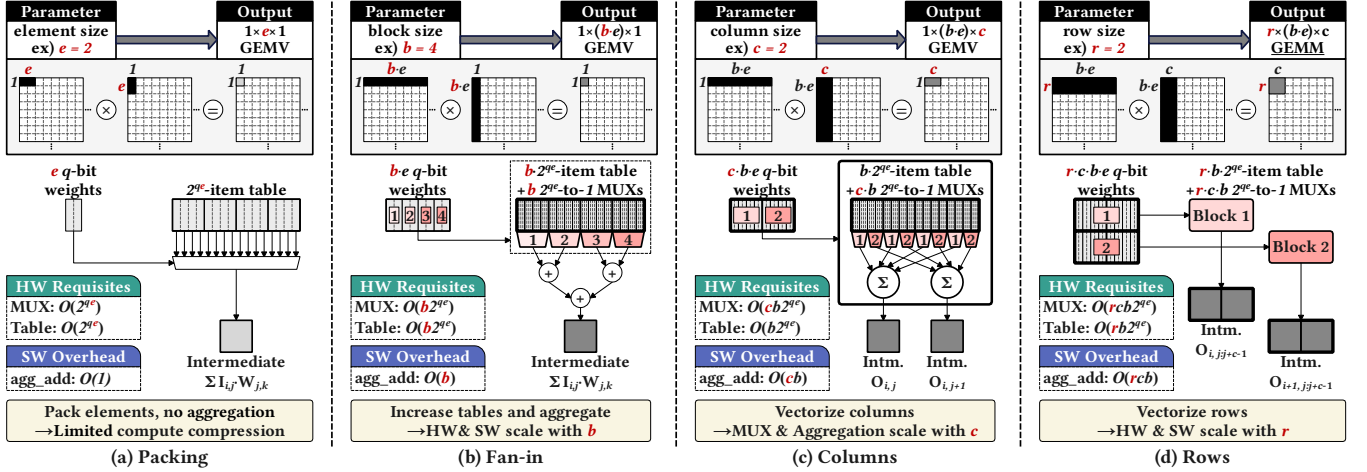


Figure 3: Design-space progression for in-register table-driven computation from GEMV to GEMM. Panels (a)–(d) expand the output dimensionality under quantization bitwidth q , by varying (a) packing e , (b) fan-in b , (c) column tile c , and (d) row tile r . Each panel reports how MUX/select logic and register-resident table entries scale, along with the extra aggregation-add software overhead (agg_add) introduced by the decomposition.

on vector CPUs by reusing most of the existing SIMD datapath and adding only minimal support for in-register table computation. At the same time, this execution style should not be tied to a single kernel organization. Instead, ExaGEMM exposes a parameterized execution space so that different quantized models and CPU targets can select appropriate operating points. We next describe the framework flow, structural execution space, SIMD-slice support, and software-visible primitives used by the generated kernels.

3.1.1 Framework Overview. Figure 2 illustrates the model-specific CPU support generation flow in ExaGEMM. The framework takes as input a quantized ML model, including per-layer bitwidth and GEMM dimensions, together with target CPU parameters such as vector-register width and register-file capacity. With per-layer modeling, the same flow applies to both uniform and inter-layer mixed precision models. The analytical explorer then enumerates candidate execution instances of the form

$$k = (e, b, r, c, u), \quad (2)$$

where (e, b, r, c) specify the structural mapping of the table-driven GEMM—packing factor, fan-in, row tile, and column tile, respectively—and u denotes the instruction-level merge factor. The analytical explorer evaluates candidate kernels $k = (e, b, r, c, u)$ using estimates of feasibility, instruction count, memory traffic, compute cost, and hardware overhead. It then uses these estimates under candidate support points to select layer-wise kernels, identify feasible support points, and report the resulting support frontier to the designer. These points are materialized as implementation artifacts, including custom ISA specs, gem5 ISA patches, and generated GEMM kernels, for subsequent system-level design-space iteration.

3.1.2 Parameterized Execution Space. Figure 3 summarizes the structural execution space exposed by ExaGEMM. The *packing factor* e determines how many low-bit symbols are packed into one table index, the *fan-in* b determines how many packed groups are aggregated within one table-driven block, the *column tile* c determines how many output columns are produced per block, and the *row tile* r determines how many output rows are generated simultaneously. Together, these parameters control the size of the

register-resident table, the complexity of the select network, and the amount of aggregation work required after table application.

The progression in Figure 3 makes the main structural tradeoff explicit. Increasing e improves packing efficiency but enlarges table cardinality through the 2^{q_e} term. Increasing b improves reuse across packed groups but raises aggregation overhead. Increasing c and r extends reuse across output columns and rows, but also increases the amount of register-resident table state and the complexity of the select network. Accordingly, no single structural mapping dominates across all bitwidths, layer shapes, and CPU resource envelopes. Compact mappings reduce aggregation and register pressure, whereas larger GEMM layers can better amortize aggressive tiling.

Figure 3 captures only the *structural* part of the execution space. Beyond (e, b, r, c) , ExaGEMM also includes the ILM factor u , a *schedule-level* parameter. While (e, b, r, c) determine how the table-driven operator is blocked across the GEMM dimensions, u determines how many K-subblocks are merged into one scheduled block. The (e, b, r, c) define the structural shape of the kernel, while u controls software-visible instruction grouping and register reuse.

3.1.3 Hardware Mapping to a SIMD Slice. Given the structural execution space above, ExaGEMM adopts a lightweight SIMD-slice support point shown in Figure 4. The key idea is to realize table-driven low-bit GEMM using *register-resident* data structures and existing SIMD arithmetic datapaths, while introducing only a lightweight select network to perform table application. This choice is motivated by the desire to keep the required hardware support CPU-native and practical for edge-oriented deployments.

As shown in Figure 4(a), the vector register file (VRF) stores the live state of execution, including the staged weight block, the register-resident table entries, and the output register. The table is generated and consumed entirely within the SIMD slice, so table application no longer depends on repeated accesses to memory-resident LUTs. Figure 4(b) summarizes the target hardware envelope in terms of vector-register (VR) count, VR width, and the available

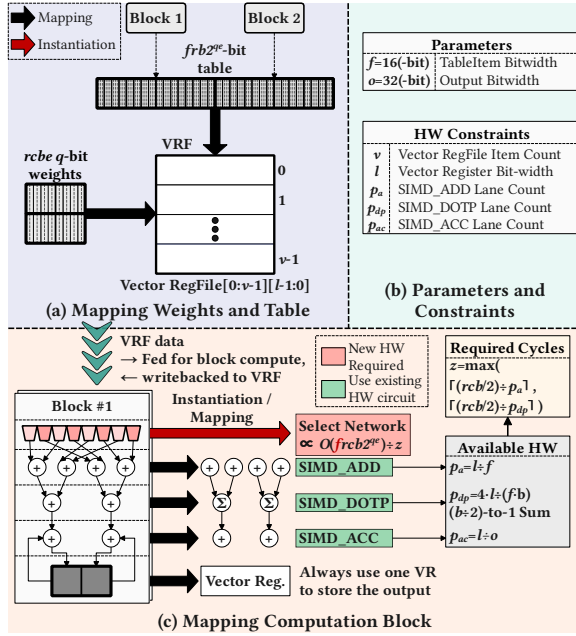


Figure 4: Mapping ExaGEMM’s table-driven block computation onto a vector-CPU SIMD slice. (a) Quantized weights and the $rb2^{q_e}$ -entry table are staged in the vector register file (VRF). (b) Parameters and hardware constraints define the available lane parallelism. (c) The computation block is realized by instantiating a select network while reusing existing SIMD ADD/DOTP/ACC units; intermediate results are reduced and written back to the VRF.

SIMD lane counts for the reused arithmetic units. These parameters define the constraints under which candidate kernels must operate.

Figure 4(c) then shows the key architectural property of ExaGEMM: The major new hardware support is a *select network*, while the existing SIMD_ADD, SIMD_DOTP, and SIMD_ACC circuits are reused for table generation, table-driven compute, and accumulation, respectively. If the required computation for each instruction exceeds the existing circuits, a multi-cycle counter controls the data feeding. This mapping keeps the support point lightweight and makes hardware overhead explicit: the select network scales with the amount of table-driven indirection that a candidate kernel requires, while the remaining computation is serviced by the existing SIMD execution pipeline. In other words, ExaGEMM does not propose a new accelerator array; it exposes a CPU-native support point that extends the SIMD slice just enough to make in-register table-driven low-bit GEMM practical.

This hardware organization naturally induces two execution primitives in ExaGEMM. The VRF-resident table is generated from packed activation state and then consumed together with a weight block to update an output tile. Table 1 summarizes the resulting table-generation and table-driven GEMM primitives.

3.1.4 Instruction Primitives and Instruction-Level Merging.

The logical primitives in Table 1 provide the interface between the lightweight SIMD ISA extension and the generated GEMM/GEMV kernels. In the baseline case, one TBL_ $r \times b \times e$ instruction generates a table for one K-subblock, and one or more GEMM_ $r \times b \times e \times c$ instances use that table to update the corresponding output tiles. However,

Table 1: Logical ISA primitives induced by the lightweight SIMD support point.

Primitive	Logical effect
TBL_ $r \times b \times e$	Consumes a packed activation block and generates an in-register table in the VRF for subsequent table-driven computation.
GEMM_ $r \times b \times e \times c$	Consumes one in-register table and one weight block to update an $r \times c$ output tile through the select network and reused SIMD datapaths.

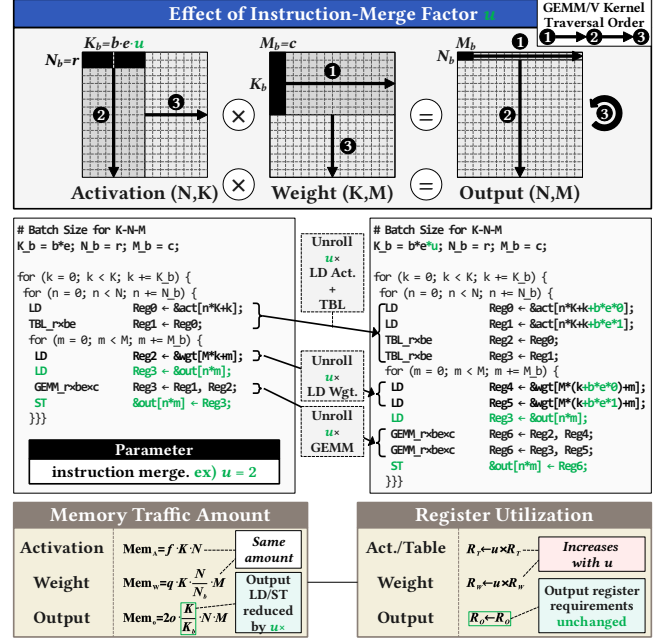


Figure 5: Effect of instruction-level merge (ILM) factor u on scheduled table-driven GEMM execution. Increasing u merges multiple K-subblocks into one scheduled block, reducing redundant output load/store traffic and loop overhead while increasing live activation/table and weight register demand. The output-register requirement remains unchanged because all merged K-subblocks accumulate into the same output object.

executing these primitives naively can introduce redundant output load/store traffic and loop overhead, especially when multiple neighboring K-subblocks contribute to the same output tile.

To address this, ExaGEMM introduces an instruction-level merge (ILM) factor u (Figure 5). Increasing u merges multiple K-subblocks into one scheduled block, so that

$$K_b = b \cdot e \cdot u, \quad N_b = r, \quad M_b = c. \quad (3)$$

Relative to the baseline case $u = 1$, the merged schedule keeps activation-derived table state live for a longer window, while reusing the same output VR across multiple GEMM_ $r \times b \times e \times c$ instances. As shown in Figure 5, this reduces redundant output LD/ST activity and loop overhead while increasing live activation/table and weight register demand. The output-register requirement remains unchanged because all merged K-subblocks still accumulate into the same output tile.

This distinction between structural and schedule-level parameters is important for ExaGEMM. The tuple (e, b, r, c) determines the table-driven block shape and associated hardware requirements, while u controls how aggressively the generated kernel amortizes output traffic and loop overhead. Together, they define the full execution candidate $k = (e, b, r, c, u)$ evaluated by the analytical explorer in the following subsection.

3.2 Analytical Exploration Framework

Given the lightweight SIMD support point and parameterized execution space in subsection 3.1, ExaGEMM evaluates which candidate kernels are feasible and worth supporting for a target quantized model. For each candidate, the analytical explorer models register feasibility, logical instruction count, memory traffic, arithmetic/-control cost, and hardware overhead. These quantities are used to prune infeasible candidates, rank feasible kernels, and report a non-dominated support frontier before system-level validation.

3.2.1 Feasibility and Cost Models. For a quantized GEMM layer ℓ of shape (N_ℓ, K_ℓ, M_ℓ) and bitwidth q_ℓ , ExaGEMM evaluates each candidate $k = (e, b, r, c, u)$ in Equation 2. Modeling q_ℓ per layer allows the same framework to cover both uniform- and mixed-quantization settings.

Register feasibility. For candidate k , the modeled VRF-resident live state consists of the register-resident table, weight block, and output tile:

$$R_T(k) = \left\lceil \frac{f r b 2^{q_\ell} u}{l} \right\rceil, \quad R_W(k) = \left\lceil \frac{q_\ell r c b e u}{l} \right\rceil, \quad R_O(k) = \left\lceil \frac{o r c}{l} \right\rceil, \quad (4)$$

where l is the vector-register width, f is the table-entry width, and o is the output / accumulation width. The factor u appears because ILM increases simultaneously live table and weight state. We define

$$U_{\text{VRF}}(k) = \frac{R_T(k) + R_W(k) + R_O(k)}{v}, \quad (5)$$

and require

$$U_{\text{VRF}}(k) \leq 1, \quad R_O(k) \leq 1. \quad (6)$$

The first constraint enforces total VRF capacity; the second enforces the single-destination-VR execution model assumed by ExaGEMM.

Instruction and traffic models. For compactness, define

$$v_\ell(k) = \left\lceil \frac{N_\ell}{r} \right\rceil, \quad \kappa_\ell(k) = \left\lceil \frac{K_\ell}{b e} \right\rceil, \quad \mu_\ell(k) = \left\lceil \frac{M_\ell}{c} \right\rceil, \quad \kappa_\ell^{(u)}(k) = \left\lceil \frac{K_\ell}{b e u} \right\rceil. \quad (7)$$

The dominant logical instruction counts are

$$I_{\text{TBL}}(\ell, k) = v_\ell(k) \kappa_\ell(k), \quad I_{\text{GEMM}}(\ell, k) = v_\ell(k) \kappa_\ell^{(u)}(k) \mu_\ell(k). \quad (8)$$

ILM changes the schedule rather than the logical work decomposition, so u does not reduce the number of TBL or GEMM primitives. Instead, it amortizes output load/store activity and loop overhead:

$$I_{\text{LD}}^o(\ell, k) = I_{\text{ST}}^o(\ell, k) = v_\ell(k) \kappa_\ell^{(u)}(k) \mu_\ell(k), \quad (9)$$

$$C_{\text{CTL}}(\ell, k) = \alpha_{\text{CTL}} v_\ell(k) \kappa_\ell^{(u)}(k) \mu_\ell(k), \quad (10)$$

where α_{CTL} is the platform-dependent control-overhead cost per scheduled block.

ExaGEMM models schedule-level memory traffic without attempting to model the full cache hierarchy. At fixed (e, b, r, c) , activation and weight traffic are invariant with respect to u , while output traffic decreases as output LD/ST is amortized across merged K-subblocks:

$$B_o(\ell, k) = o r c (I_{\text{LD}}^o(\ell, k) + I_{\text{ST}}^o(\ell, k)), \quad (11)$$

$$B_{\text{mem}}(\ell, k) = B_a(\ell, k) + B_w(\ell, k) + B_o(\ell, k), \quad (12)$$

where B_a and B_w denote activation and weight traffic under the selected structural mapping.

Execution cost and hardware overhead. Let $\tau_{\text{TBL}}(k)$ and $\tau_{\text{GEMM}}(k)$ denote the per-instruction costs of the two logical primitives. Because GEMM reuses the SIMD_ADD, SIMD_DOTP, and SIMD_ACC

datapaths, its per-instruction cost is bounded by the slowest reused arithmetic component:

$$\tau_{\text{GEMM}}(k) = \max(\tau_{\text{ADD}}(k), \tau_{\text{DOTP}}(k), \tau_{\text{ACC}}(k)). \quad (13)$$

The arithmetic/control-side estimate is

$$C_{\text{cmp}}(\ell, k) = I_{\text{TBL}}(\ell, k) \tau_{\text{TBL}}(k) + I_{\text{GEMM}}(\ell, k) \tau_{\text{GEMM}}(k) + I_{\text{agg}}(\ell, k) \tau_{\text{ADD}}(k) + C_{\text{CTL}}(\ell, k), \quad (14)$$

where I_{agg} denotes the aggregation-add work by the structural mapping in Figure 3. C_{cmp} captures arithmetic/control work only, while memory-side effects enter the final ranking objective by B_{mem} .

To rank candidates, ExaGEMM combines arithmetic/control cost and schedule-level traffic into a single analytical execution cost:

$$C_{\text{exe}}(\ell, k) = C_{\text{cmp}}(\ell, k) + \alpha_{\text{mem}} B_{\text{mem}}(\ell, k), \quad (15)$$

where α_{mem} is a platform-dependent traffic-to-cost factor that maps the schedule-level traffic proxy B_{mem} to an effective execution-cost penalty. α_{mem} is obtained by calibrating the analytical score against vectorized load/store microbenchmarks that sweep working-set sizes across the cache and memory hierarchy. C_{exe} is the final execution-side ranking objective used during exploration. It is not intended to predict the full memory hierarchy exactly; rather, it provides a consistent efficiency ordering before detailed simulation.

Finally, hardware overhead is modeled per supported primitive configuration $p = (e, b, r, c, q)$ as

$$H_{\text{sel}}(p) = \alpha_{\text{sel}} S_{\text{sel}}(e, b, r, c, q), \quad (16)$$

where S_{sel} is the select-network scaling term corresponding to Figure 3, and α_{sel} is a technology-dependent factor that maps S_{sel} to normalized selector overhead using ASIC synthesis under the target process design kit.

Taken together, Equation 6–Equation 16 define a compact analytical surrogate for model-specific ISA support exploration. ExaGEMM uses this surrogate both to rank candidate kernels and to identify lightweight support points worth materializing for subsequent gem5-based evaluation.

3.2.2 Kernel Selection and Support Ranking. Using the models above, ExaGEMM performs exploration at two levels: (i) layer-wise kernel selection and (ii) model-level support-point ranking.

Layer-wise kernel selection. For each support point σ and layer ℓ , ExaGEMM enumerates the candidate set $\mathcal{K}_\ell(\sigma)$ over the supported ranges of (e, b, r, c, u) . Candidates violating Equation 6 are pruned. Among the remaining candidates, ExaGEMM selects

$$k_\ell^*(\sigma) = \arg \min_{k \in \mathcal{K}_\ell(\sigma)} C_{\text{exe}}(\ell, k) \quad \text{s.t. } U_{\text{VRF}}(k) \leq 1, \quad R_O(k) \leq 1, \quad (17)$$

where C_{exe} is defined in Equation 15. Hardware overhead is a property of the support point rather than of an individual layer mapping, so layer-wise selection minimizes execution-side cost under feasibility constraints, while the efficiency–hardware tradeoff is resolved at the support-point level.

Support-point frontier. A custom ISA support point σ is defined as a set of distinct supported primitive configurations $p = (e, b, r, c, q)$. The ILM factor u is not part of σ because it changes the software schedule but does not require a different primitive. For each σ , ExaGEMM restricts layer-wise selection to compatible candidates and discards the support point if any layer has no feasible

mapping. Otherwise, the selected per-layer kernels are aggregated into model-level execution cost and hardware overhead:

$$C_{\text{model}}(\sigma) = \sum_{\ell} C_{\text{exe}}(\ell, k_{\ell}^*(\sigma)), \quad (18)$$

$$H_{\text{model}}(\sigma) = \sum_{p \in \sigma} H_{\text{sel}}(p), \quad (19)$$

where p denotes one supported configuration in the support point. Equation 19 uses an additive support-cost model; if multiple configurations share hardware, it should be interpreted as a conservative ranking proxy. For completeness, ExaGEMM may still report

$$B_{\text{model}}(\sigma) = \sum_{\ell} B_{\text{mem}}(\ell, k_{\ell}^*(\sigma)), \quad (20)$$

but B_{model} is diagnostic only, since traffic is already incorporated into C_{model} through Equation 15.

ExaGEMM reports the non-dominated feasible support points over the efficiency–hardware tradeoff, leaving the final choice to the designer. Shortlisted points are then materialized as ISA specifications, gem5 ISA-model patches, and generated GEMM kernels for system-level validation. Algorithm 1 summarizes the model-level exploration procedure.

Algorithm 1 Model-level support exploration in ExaGEMM

Require: Quantized model $\mathcal{L}$, target CPU parameters, support-point set Σ
Ensure: Non-dominated feasible support points

```

1: for each  $\sigma \in \Sigma$  do
2:   feasible  $\leftarrow$  true
3:   for each layer  $\ell \in \mathcal{L}$  do
4:     Enumerate  $\mathcal{K}_{\ell}(\sigma)$ 
5:     Prune candidates violating Equation 6
6:     if  $\mathcal{K}_{\ell}(\sigma) = \emptyset$  then
7:       feasible  $\leftarrow$  false
8:     break
9:   end if
10:  Select  $k_{\ell}^*(\sigma)$  using Equation 17
11: end for
12: if feasible then
13:   Aggregate  $C_{\text{model}}(\sigma)$  and  $H_{\text{model}}(\sigma)$ 
14:   Optionally record  $B_{\text{model}}(\sigma)$ 
15: end if
16: end for
17: Extract the non-dominated feasible points over  $(C_{\text{model}}, H_{\text{model}})$ 
18: Return the resulting support frontier to the designer

```

This procedure reduces the model-specific support space to a small set of feasible, non-dominated candidates for designer-guided selection and subsequent gem5-based evaluation.

4 Evaluation

We evaluate ExaGEMM from both CAD and system perspectives by asking: (1) whether the analytical explorer prunes the search space while preserving strong candidates, (2) whether realistic workloads induce different feasible and execution-favorable primitive-side landscapes under fixed target constraints, (3) whether frontier-selected support points and kernel schedules improve end-to-end performance under modest hardware overhead.

4.1 Experimental Setup

Table 2 and 3 summarize the target platforms and evaluated workloads. We study representative x86 and ARM vector CPU targets on gem5-AVX [2, 11] v20.1.0.0, spanning 128/256/512-bit SIMD

Table 2: Representative target vector-CPU configurations used in gem5.

Attribute	Workstation	Laptop	Mobile
CPU model	Ryzen 9 9950X	Ryzen 7 6800U	Cortex-A78
Simulation mode		DerivO3CPU	
ISA	x86-64	x86-64	ARMv8-A
Cores	16	8	4
Frequency	5.7 GHz	4.7 GHz	3.0 GHz
L1 I / D	32 KB / 48 KB	32 KB / 32 KB	64 KB / 64 KB
L2 cache	1 MB/core	512 KB/core	512 KB/core
L3 cache	64 MB shared	16 MB shared	4 MB shared
DRAM	DDR5-6400	DDR5-4800	LPDDR5-6400
VR param.s l / v	512 b / 32	256 b / 16	128 b / 16

Table 3: Model targets used in the evaluation.

Model	Modality	Label	Quantization setting
DeiT-B	Vision	A	W1A8 uniform
Llama2-7B	Language	B	W2A16 uniform
Llama2-7B	Language	C	W{1,2,4}A16 mixed

configurations with different register budgets. GEMM tiles are distributed across all available cores along the output-row dimension; since each core’s in-register table is generated and consumed locally, inter-core coherence overhead is minimized. The workloads span two uniform-quantization cases (DeiT-B W1A8, Llama2-7B W2A16), and one mixed-precision configuration (Llama2-7B with AMQ-style inter-layer W1,2,4A16 [12]), exploring workload-aware support selection across quantization regimes. For each target, ExaGEMM explores candidate kernels $k = (e, b, r, c, u)$, materializes selected frontier points, and evaluates them via model-level support-tradeoff analysis and end-to-end comparison against representative baselines. The analytical surrogate is calibrated per target/process node: α_{mem} is obtained from gem5 microbenchmarks, and α_{sel} is derived from ASIC synthesis of the selector-network and full SIMD-datapath RTL designs in a TSMC 28 nm process at 1 GHz using Cadence Genus 21.10. Area and power are reported at tt0p9v25c.

4.2 Explorer Effectiveness and Fidelity

We evaluate the analytical explorer along two axes: (1) how aggressively ExaGEMM reduces the candidate space before simulation, and (2) whether the analytical ranking metric preserves the ordering needed for shortlist formation.

4.2.1 Search Space Reduction. Figure 6 shows that ExaGEMM shrinks large candidate spaces to small frontiers before gem5 runs. For uniform quantization, DeiT-B W1A8 shrinks from 780/1170/1638 enumerated primitive candidates to 10/11/14 frontier points on the 128/256/512-bit targets, corresponding to 98.7%/99.1%/99.1% reduction. Llama2-7B W2A16 similarly shrinks from 630/1050/1575 candidates to 7/9/12 frontier points, i.e., 98.9%/99.1%/99.2% reduction.

For mixed precision, ExaGEMM applies the same idea hierarchically. It first computes per-quantization primitive frontiers, forms support-set candidates only from those reduced spaces, and then applies a model-level Pareto filter. For the Llama2-7B W{1,2,4}A16, the raw space contains 187,813,080/869,505,000/2,934,579,375 structurally enumerated support-set combinations on the 128/256/512-bit targets. After per-quantization frontier reduction, this shrinks to 280/490/1560 candidates, and the final frontier retains only 14–16 sets per target, yielding 12,520,872×/57,967,000×/183,411,211× reduction before gem5. Figure 7 presents the 256-bit case: only a small subset of the reduced combinations remains Pareto-competitive.

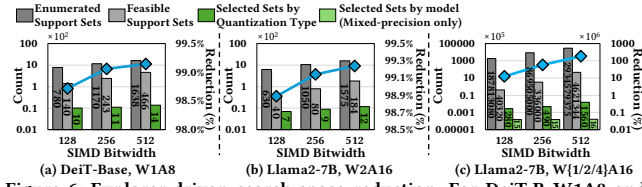


Figure 6: Explorer-driven search-space reduction. For DeiT-B W1A8 and Llama2-7B W2A16, bars show enumerated primitives, VRF-feasible primitives, and retained frontier points. For Llama2-7B W{1/2/4}A16, bars show the hierarchical mixed-precision flow: enumerated support sets, post-per-quantization candidate sets, and final model-level frontier sets. The line reports overall reduction from the fully enumerated space.

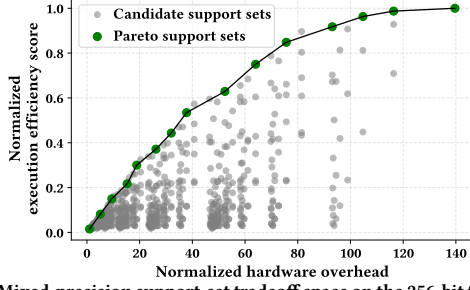


Figure 7: Mixed-precision support-set tradeoff space on the 256-bit target. Each point denotes a candidate mixed-precision support set formed by combining Pareto-efficient primitive points from the individual quantization regimes for Llama2-7B W{1/2/4}A16. Highlighted points form the final model-level Pareto shortlist retained for materialization and gem5 validation. Execution efficiency is defined as the inverse of execution cost.

4.2.2 Modeling Fidelity. We evaluate ranking fidelity as whether the modeled execution cost (C_{exe}) preserves the ordering needed to screen candidates before detailed simulation. For each workload-target pair, we compare the C_{exe} against gem5 latency on a sample set comprising all analytically retained Pareto points together with randomly sampled feasible points, for a total of 30 points per pair. As shown in Figure 8, the Spearman correlations are 0.945/0.982/0.969 for DeiT-B W1A8 and 0.889/0.951/0.952 for mixed-precision Llama2 on the 512/256/128-bit targets.

We also calibrate the modeled hardware cost with ASIC synthesis results on the augmented SIMD datapath RTL for the Llama2-7B W{1,2,4}A16 Pareto support sets across the 128/256/512-bit targets. Figure 9 compares modeled costs with area and power overheads, showing near-linear correlation. The modeled cost, normalized to the smallest synthesized support point’s area/power, tracks synthesis closely, with a mean absolute percentage error (MAPE) of 1.4% for area and 6.3% for power. Relative to the original full SIMD datapath baseline, retained support points incur 0.049–1.714% / 0.083–2.731% area/power overhead on 128-bit, 0.024–3.564% / 0.042–5.448% on 256-bit, and 0.381–13.21% / 0.607–20.81% on 512-bit, confirming that the added select/feed support remains affordable.

Together, these results make ExaGEMM effective for pre-silicon screening, preserving candidate ordering for high-value shortlist formation while estimating hardware cost accurately enough to compare support points before expensive gem5 evaluation.

4.3 Primitive-Side Efficiency Landscapes

We examine the primitive-side execution efficiency landscape under fixed constraints before hardware cost is introduced. Figure 10 maps the relative analytical score over effective K-block size $K_b = b \cdot e$ and column tile c . For each workload-target pair, the color at each point

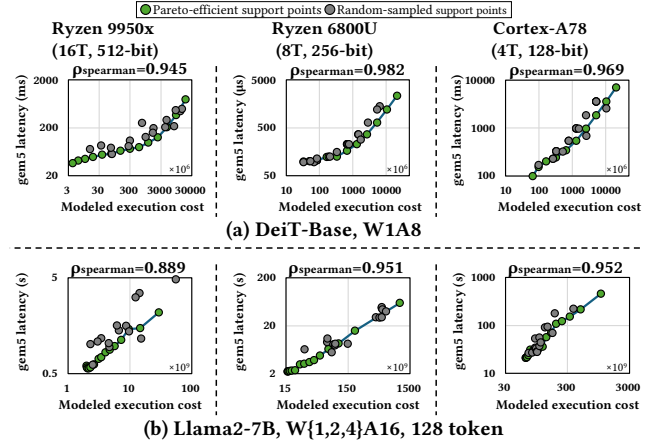


Figure 8: Ranking fidelity of the analytical explorer. Each panel compares the modeled execution cost with gem5-measured latency for analytically retained Pareto points and randomly sampled feasible points. Spearman correlations of 0.889–0.982 indicate strong ordering fidelity.

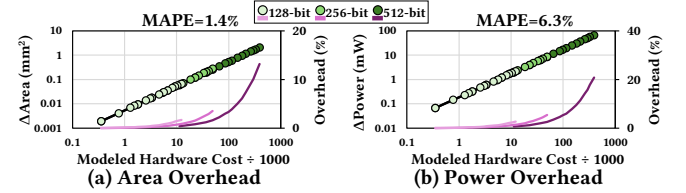


Figure 9: Hardware-cost calibration against TSMC 28 nm synthesis at 1GHz for Llama2-7B W{1,2,4}A16 support sets on 128/256/512-bit targets. Each point compares the modeled hardware cost with the synthesized area/power overhead for a single support point.

reports the best relative analytical score attainable at fixed (K_b, c) after optimizing over the remaining kernel parameters (r, u) among feasible candidates. Equivalently, the plotted value is the normalized inverse execution cost, $C_{\text{best}}/C_{\text{best}@}(K_b, c)$, where $C_{\text{best}@}(K_b, c)$ is the best modeled execution cost at that fixed primitive shape. The highlighted cells thus indicate feasible execution-favorable operating points rather than unconstrained primitive optima.

Two patterns emerge. First, the two uniform workloads, DeiT-B W1A8 and Llama2-7B W2A16, each exhibit a single dominant hotspot once the target width is fixed. As SIMD support widens, this hotspot shifts toward larger K_b and c , indicating that the high-performance region is compact and largely target-scaled.

Second, the mixed-precision Llama2-7B W{1,2,4}A16 workload does not collapse to one hotspot. Instead, it exhibits three separated dominant regions aligned with the active quantization regimes, labeled W4, W2, and W1 in Figure 10. Across the 128/256/512-bit targets, these regime-specific hotspots move systematically toward larger K_b and c . Thus, the mixed-precision workload is shaped not by arbitrary layer-to-layer noise, but by the superposition of several regime-specific primitive pressures within one model.

Selector overhead is introduced only when these primitive choices are aggregated into model-level support points. Thus, Figure 10 isolates the primitive-side demand pattern in the execution space: the high-performance region is compact for the uniform workloads but split across several regime-specific hotspots for mixed-precision Llama2. We next ask whether this richer primitive-side structure is large enough to change the model-level support decision once hardware cost is included.

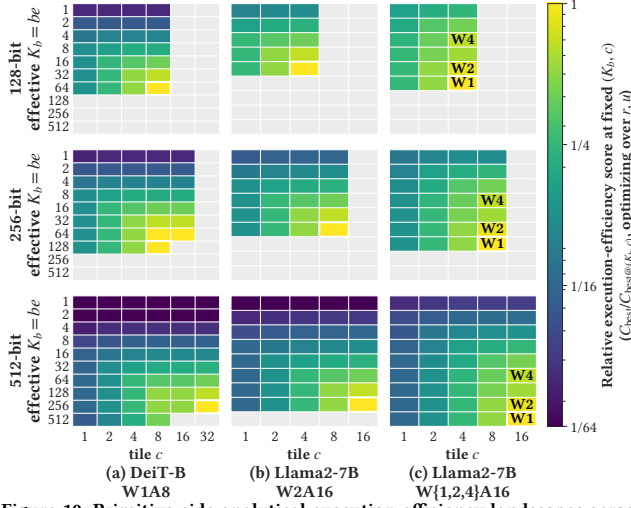


Figure 10: Primitive-side analytical execution efficiency landscapes across workloads and targets. For each workload-target pair, the heatmap color at each (K_b, c) reports the best relative analytical score at fixed effective K-block size $K_b = be$ and column tile c after optimizing over the remaining kernel parameters (r, u) among feasible candidates; highlighted cells mark the best (K_b, c) value(s). Uniform DeiT-B W1A8 and uniform Llama2-7B W2A16 each exhibit a compact target-scaled hotspot, whereas mixed-precision Llama2-7B W{1,2,4}A16 exhibits three regime-specific hotspots labeled W4/W2/W1.

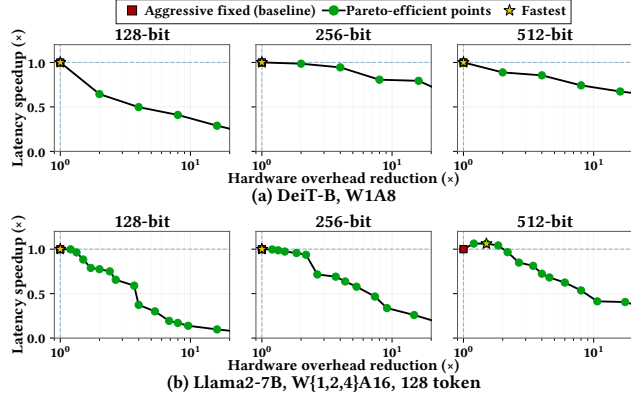


Figure 11: Measured support-tradeoff frontier normalized to the aggressive fixed baseline. For each workload/target pair, the horizontal axis reports hardware overhead reduction relative to the aggressive fixed design, and the vertical axis reports latency speedup relative to that same baseline. DeiT-B stays close to the aggressive fixed point, whereas mixed-precision Llama2 exhibits a richer frontier, especially on the 512-bit target.

4.4 End-to-End Comparative Evaluation

Finally, we evaluate whether the support points selected by ExaGEMM remain attractive after gem5 materialization. This closes the design loop of ExaGEMM by asking which primitive-side execution-favorable regions remain worth implementing once hardware overhead is included. Figure 11 compares measured frontier points against an *aggressive fixed* baseline, i.e., the feasible single-primitive design with the highest normalized support cost on each target, while Figure 12 reports end-to-end performance against external baselines.

Frontier value depends on workload. In Figure 11, the aggressive fixed point for DeiT-B W1A8 lies on the measured frontier across all three targets, indicating that a single aggressive primitive is already a competitive support choice for the uniform workload. In

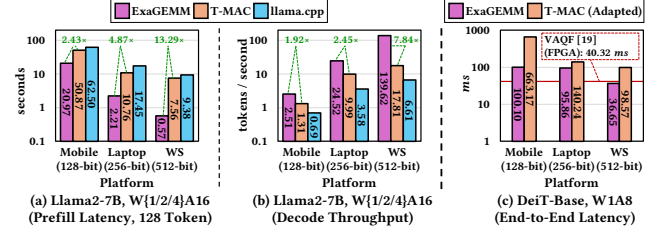


Figure 12: End-to-end performance of ExaGEMM against external baselines across the 128/256/512-bit targets. (a) Llama2-7B W{1,2,4}A16 prefill latency at 128 tokens. (b) Llama2-7B W{1,2,4}A16 decode throughput. (c) DeiT-B W1A8 end-to-end latency. AVX512-specific adaptations are applied to T-MAC and llama.cpp. ExaGEMM outperforms the software baselines, with much larger widening gains on Llama than on DeiT-B.

contrast, Llama2-7B W{1,2,4}A16 exhibits a much richer frontier; on the 512-bit target, a measured frontier point is simultaneously faster and lower-cost than the aggressive fixed baseline. Thus, exploration still recovers the aggressive fixed design when it is Pareto-efficient, but the value of workload-aware frontier selection is much larger for mixed-precision LLM workloads than for DeiT-B.

Widening benefits LLMs much more than ViTs. This difference is reinforced by Figure 12, comparing ExaGEMM to the prior SIMD kernels [5, 22]. For **Llama2-7B W{1,2,4}A16**, ExaGEMM reduces **prefill latency** by 2.43–13.29 $\times$ and improves **decode throughput** by 1.92–7.84 $\times$. For **DeiT-B W1A8**, ExaGEMM reduces **latency** by 1.46–6.62 $\times$ over T-MAC. Although T-MAC was originally proposed for LLMs, its GEMM kernel can be adapted to low-bit ViT variants.

The widening payoff is similarly asymmetric. Moving from 256-bit to 512-bit support reduces Llama prefill latency by 3.89 $\times$ and increases decode throughput by 5.69 $\times$, whereas DeiT-B improves by only 2.62 $\times$. DeiT-B also changes little from 128- to 256-bit, while Llama already improves sharply over that range. This is the end-to-end counterpart of Figure 10: DeiT-B is comparatively uniform, whereas mixed-precision Llama exposes a broader set of profitable primitive shapes and therefore benefits more from wider SIMD support and workload-aware selection.

Overall, Figure 11 and Figure 12 show that a single aggressive primitive can be nearly sufficient for the uniform DeiT-B workload studied here, whereas for mixed-precision LLM workloads, workload-aware frontier selection changes the support decision itself and translates into substantial end-to-end gains.

5 Conclusion

ExaGEMM formulates lightweight ISA support for low-bit CPU GEMM as a pre-silicon support-selection problem. By reusing existing SIMD datapaths for table generation and accumulation, it reduces new hardware to an in-register select network and treats hardware overhead as an explicit design constraint. ExaGEMM prunes large candidate spaces before gem5, identifies compact non-dominated support frontiers, and generates implementation artifacts for system-level validation. The key result is workload-dependent: a single aggressive primitive is nearly sufficient for uniform DeiT-B, whereas mixed-precision Llama2 benefits substantially from workload-aware support selection. Across x86 and ARM targets, ExaGEMM delivers substantial gains over software-only and fixed-support baselines, providing a practical path to workload-aware low-bit CPU support across diverse quantization regimes.

References

- [1] Arm Ltd. [n. d.]. A64 SIMD Vector Instructions. <https://developer.arm.com/documentation/100069/0609/A64-SIMD-Vector-Instructions>. Accessed: March 26, 2025.
- [2] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. 2011. The gem5 Simulator. *ACM SIGARCH Comput. Archit. News* 39, 2 (Aug. 2011), 1–7. doi:10.1145/2024716.2024718
- [3] Razvan-Gabriel Dumitru, Vikas Yadav, Rishabh Maheshwary, Paul-Ioan Clotan, Sathwik Tejaswi Madhusudhan, and Mihai Surdeanu. 2024. Layer-Wise Quantization: A Pragmatic and Effective Method for Quantizing LLMs Beyond Integer Bit-Levels. arXiv:2406.17415 [cs.CL]. <https://arxiv.org/abs/2406.17415>
- [4] Razvan-Gabriel Dumitru, Vikas Yadav, Rishabh Maheshwary, Paul Ioan Clotan, Sathwik Tejaswi Madhusudhan, and Mihai Surdeanu. 2025. Variable Layerwise Quantization: A Simple and Effective Approach to Quantize LLMs. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 534–550. doi:10.18653/v1/2025.findings-acl.29
- [5] Georgi Gerganov and ggml-org Contributors. 2026. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>. <https://github.com/ggml-org/llama.cpp> Accessed: 2026-01-18.
- [6] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. arXiv:2103.13630 [cs.CV]. <https://arxiv.org/abs/2103.13630>
- [7] Yuanhong He, Peiyu Niu, Jun Chen, Chenchen Zhang, and Chao Yang. 2026. W4A16 Mixed-Precision Matrix Multiplication on Decoupled Architecture: Kernel Design and Memory Bottleneck Analysis for Ascend NPUs. arXiv:2601.16536 [cs.DC]. <https://arxiv.org/abs/2601.16536>
- [8] Huanqi Hu, Bowen Xiao, Shixuan Sun, Jianian Yin, Zhexi Zhang, Xiang Luo, Chengquan Jiang, Weiqi Xu, Xiaoying Jia, Xin Liu, and Minyi Guo. 2025. LiquidGEMM: Hardware-Efficient W4A8 GEMM Kernel for High-Performance LLM Serving. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. Association for Computing Machinery, New York, NY, USA, 1619–1630. doi:10.1145/3712285.3759852
- [9] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- [10] Yongkweon Jeon, Chungman Lee, Eulrang Cho, and Yeonju Ro. 2022. Mr.BiQ: Post-Training Non-Uniform Quantization Based on Minimizing the Reconstruction Error. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 12329–12338.
- [11] Seungmin Lee, Youngsok Kim, Dukyun Nam, and Jong Kim. 2024. Gem5-AVX: Extension of the Gem5 Simulator to Support AVX Instruction Sets. *IEEE Access* 12 (2024), 20767–20778. doi:10.1109/ACCESS.2024.3359296
- [12] Sangjun Lee, Seung-taek Woo, Jun-gyu Jin, Changhun Lee, and Eunhyeok Park. 2025. AMQ: Enabling AutoML for Mixed-precision Weight-Only Quantization of Large Language Models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 35520–35538.
- [13] Ju-Hung Li, Jih-Kuan Lin, Yung-Cheng Su, Chi-Wei Chu, Lai-Tak Kuok, Hung-Ming Lai, Chao-Lin Lee, and Jenq-Kuen Lee. 2023. SIMD Everywhere Optimization from ARM NEON to RISC-V Vector Extensions. *arXiv preprint arXiv:2309.16509* (2023).
- [14] Xiangyu Li, Chengyu Yin, Weijun Wang, Jianyu Wei, Ting Cao, and Yunxin Liu. 2025. Vec-LUT: Vector Table Lookup for Parallel Ultra-Low-Bit LLM Inference on Edge Devices. arXiv:2512.06443 [cs.DC]. <https://arxiv.org/abs/2512.06443>
- [15] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *MLSys*.
- [16] Hyunwoo Oh, KyungIn Nam, Rajat Bhattacharjya, Hanning Chen, Tamoghno Das, Sanggeon Yun, Suyeon Jang, Andrew Ding, Nikil Dutt, and Mohsen Imani. 2025. T-SAR: A Full-Stack Co-design for CPU-Only Ternary LLM Inference via In-Place SIMD ALU Reorganization. arXiv:2511.13676 [cs.AR]. <https://arxiv.org/abs/2511.13676>
- [17] ONNX Runtime Developers. 2026. ONNX Runtime Execution Providers. <https://onnxruntime.ai/docs/execution-providers/>. Accessed: 2026-01-18.
- [18] Mariam Rakka, Mohammed E. Fouda, Pramod Khargonekar, and Fadi Kurdahi. 2024. A Review of State-of-the-art Mixed-Precision Neural Network Frameworks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 12 (2024), 7793–7812. doi:10.1109/TPAMI.2024.3394390
- [19] Mengshu Sun, Haoyu Ma, Guoliang Kang, Yifan Jiang, Tianlong Chen, Xiaolong Ma, Zhangyang Wang, and Yanzhi Wang. 2022. VAQF: Fully Automatic Software-Hardware Co-Design Framework for Low-Bit Vision Transformer. arXiv:2201.06618 [cs.LG]. <https://arxiv.org/abs/2201.06618>
- [20] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. 2023. BitNet: Scaling 1-Bit Transformers for Large Language Models. *arXiv preprint arXiv:2310.11453* (2023).
- [21] Xubin Wang, Zhiqing Tang, Jianxiong Guo, Tianhui Meng, Chenhao Wang, Tian Wang, and Weijia Jia. 2025. Empowering Edge Intelligence: A Comprehensive Survey on On-Device AI Models. *ACM Comput. Surv.* 57, 9, Article 228 (April 2025), 39 pages. doi:10.1145/3724420
- [22] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2025. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. In *Proceedings of the Twentieth European Conference on Computer Systems (Rotterdam, Netherlands) (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 278–292. doi:10.1145/3689031.3696099
- [23] He Xiao, Qingyao Yang, Dirui Xie, Wendong Xu, Zunhai Su, Runming yang, Wenying Zhou, Haobo Liu, Zhengwu Liu, and Ngai Wong. 2025. Exploring Layer-wise Information Effectiveness for Post-Training Quantization in Small Language Models. arXiv:2508.03332 [cs.LG]. <https://arxiv.org/abs/2508.03332>
- [24] Tianyi Zhang, Jonah Yi, Bowen Yao, Zhaozhao Xu, and Anshumali Shrivastava. 2024. NoMAD-attention: efficient LLM inference on CPUs through multiply-add-free attention. In *Proceedings of the 38th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '24)*. Curran Associates Inc., Red Hook, NY, USA, Article 3581, 25 pages.
- [25] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Sizhe Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-Bit Quantization for Efficient and Accurate LLM Serving. In *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. De Sa (Eds.), Vol. 6. 196–209. https://proceedings.mlsys.org/paper_files/paper/2024/file/5ed57c05c81d04beb716ef1d542fe9e-Paper-Conference.pdf
- [26] Yue Zheng, Yuhao Chen, Bin Qian, Xiufang Shi, Yuanhao Shu, and Jiming Chen. 2025. A Review on Edge Large Language Models: Design, Execution, and Applications. *ACM Comput. Surv.* 57, 8, Article 209 (March 2025), 35 pages. doi:10.1145/3719664